

ReviewAgent

An Agentic Pipeline for App Review Triage,
Resolution, and Response Generation
with Human-in-the-Loop Alignment

Research Proposal (Revised per Advisor Feedback)

Prepared by: Fabiha Jalal
Date: March 2026

1. Problem Statement

Mobile app stores receive millions of user reviews daily. These reviews are a rich but highly noisy source of actionable information -- containing bug reports, feature requests, performance complaints, and usability feedback buried within informal, unstructured, and often vague text (e.g., 'app keeps crashing', 'trash app 1 star', 'login broken on my samsung fix it!!!').

The Core Problem is Threefold:

- 1. Manual triage is slow and inconsistent.** Developers must manually read thousands of reviews to identify actionable issues. This process is labor-intensive, subjective, and does not scale -- leading to delayed responses, missed critical bugs, and inconsistent prioritization across team members.
- 2. No standardized issue representation exists.** Even when reviews are classified (e.g., as 'bug report' or 'feature request'), the output remains unstructured text -- not a standardized, actionable issue specification that a developer or automated system can act on. There is no established method for converting noisy review clusters into GitHub-issue-quality specifications with reproduction steps, severity, affected components, and environment details.
- 3. Existing works operate in silos.** Classification papers stop at labeling reviews. Response generation papers (RRGen, CoRe) ignore the actual underlying issue. Agentic SE papers (SWE-Agent, RepairAgent) assume well-structured GitHub issues as input. KG papers summarize but don't produce actionable triage. RLHF papers have not been applied to the app review domain.

The critical missing piece identified across all five research areas: review-to-issue translation -- converting noisy, vague, non-technical app store reviews into structured, actionable issue specifications that can drive both automated resolution and informed user responses.

2. Key Contributions (What Makes This Proposal New)

ReviewAgent introduces four novel components that collectively address the problem statement. No prior work combines these elements into a unified system:

Contribution 1: Standardized Issue Schema Mapping

A unified framework that transforms noisy review clusters into structured, standardized issue specifications with fixed fields for issue type, reproduction steps, severity, affected component, environment, and priority score. Each issue type follows established SE taxonomies: bug reports use the Zimmermann template, feature requests use user stories, performance complaints map to ISO/IEC 25010 NFR categories, usability issues align with Nielsen's heuristics, and compatibility issues generate device-OS matrices. No existing work produces standardized, developer-actionable issue specs from raw reviews.

Contribution 2: Human-in-the-Loop at Critical Decision Points

Three strategically placed HITL checkpoints grounded in selective prediction theory (Geifman & El-Yaniv, 2017): (1) post-classification verification for ambiguous reviews, (2) expert rubric-based validation of generated issue specs before downstream processing, and (3) dual-objective feedback on final responses. Human corrections propagate backward into the training loop. Prior HITL approaches are limited to end-stage feedback; by embedding oversight at high-uncertainty junctures, the system achieves complementary human-AI performance (Bansal et al., 2019) that exceeds either party alone.

Contribution 3: Knowledge Graph for Aggregation and Prioritization

A three-layer KG framework (graph construction, hierarchical clustering, schema mapping) that deduplicates, clusters, and prioritizes reviews using graph centrality (PageRank/betweenness). Reviews are grouped by aspect, sub-clustered by specific complaint, and mapped to the standardized issue schema. Existing KG approaches focus on summarization, not actionable triage. The centrality-based prioritization ensures developers see the most impactful issues first, reducing triage effort.

Contribution 4: Issue-Spec-Aware Response Generation with Dual-Objective RLHF

A response generation module explicitly aware of the structured issue spec from Stage 3, producing responses that reference the specific issue and its status rather than generic templates. Optimized via dual-objective RLHF (CMDP-grounded): maximize quality while constraining policy compliance, progressing from KTO to DPO to Constrained PPO. Existing response generators operate without structured issue knowledge, and RLHF has not been applied to this domain.

Contribution 5: Gold-Standard Benchmark Dataset

The first dataset pairing app review clusters with expert-written structured issue specifications (200-300 clusters, 3+ annotators), with inter-annotator agreement measured via Krippendorff's alpha. No such benchmark exists, enabling reproducible evaluation.

3. Theoretical Foundations

The ReviewAgent pipeline is grounded in three complementary theoretical frameworks that provide formal justification for the architecture, the placement of human-in-the-loop checkpoints, and the dual-objective optimization strategy.

Framework 1: Information Extraction Cascade Theory

The pipeline follows the information extraction cascade model (Hearst, 1999; Sarawagi, 2008), which formalizes the progressive transformation of unstructured text into structured, actionable knowledge: Raw text $\rightarrow$ Entities $\rightarrow$ Relations $\rightarrow$ Structured knowledge $\rightarrow$ Downstream task. In ReviewAgent: Reviews (raw text) $\rightarrow$ Aspects/Entities (Stage 1) $\rightarrow$ KG relations (Stage 2) $\rightarrow$ Issue specs (Stage 3) $\rightarrow$ Responses (Stage 4b). Each stage monotonically reduces entropy and increases actionability, providing a formal basis for why progressive structuring outperforms end-to-end models that skip intermediate representations.

Framework 2: Human-AI Complementary Decision Making

The HITL checkpoints are grounded in human-AI complementarity theory (Kamar, 2016; Bansal et al., 2019) and selective prediction (Geifman & El-Yaniv, 2017). The system defers to humans when confidence falls below a calibrated threshold, creating a human-AI team whose accuracy exceeds either party alone. This provides the theoretical guarantee that selective prediction with calibrated scores provably reduces the error rate compared to full automation, at human effort proportional to the uncertainty fraction of inputs.

Framework 3: Constrained Markov Decision Process (CMDP)

The dual-objective RLHF is formalized as a constrained MDP (Altman, 1999; Dai et al., 2023): Maximize $R_quality(response)$ subject to $C_compliance(response) \leq threshold$. Single-objective RLHF conflates quality and compliance into one reward signal, collapsing two distinct optimization surfaces. Dual-objective formulation respects the Pareto frontier -- finding responses that are maximally helpful without violating compliance constraints. Constrained PPO is the solver that navigates this frontier.

4. Research Questions

RQ1: Review-to-Issue Translation Quality

How accurately can an LLM-based agent translate noisy, unstructured app review clusters into structured, taxonomy-grounded issue specifications, and how do these compare to human-written GitHub issues in terms of completeness, accuracy, actionability, specificity, and clarity?

RQ2: Coupled Resolution and Response Generation

Does coupling knowledge-graph-based issue prioritization with issue-spec-aware response generation produce more specific, actionable, and helpful user responses compared to context-unaware baselines (e.g., RRGGen, CoRe)?

RQ3: Dual-Objective RLHF Effectiveness

Does dual-objective RLHF (optimizing for both response quality and policy compliance) with dimension-level expert feedback outperform single-objective preference tuning in generating safe, helpful, and accurate app review responses over iterative retraining cycles?

5. Research Objective and Specific Aims

Objective

To design, implement, and evaluate an end-to-end agentic pipeline, ReviewAgent, that automatically triages noisy app store reviews, translates them into structured and actionable issue specifications, and generates resolution-aware user responses, all refined through human-in-the-loop feedback at critical decision points and grounded in established theoretical frameworks (IE cascade, human-AI complementarity, CMDP).

Aim 1: Develop a Unified Clustering-to-Issue Translation Framework

Design a three-layer framework (knowledge graph construction, hierarchical clustering, and standardized schema mapping) that converts raw app reviews into structured, taxonomy-grounded issue specifications covering bug reports (Zimmermann template), feature requests (user stories), performance complaints (ISO/IEC 25010 NFRs), usability issues (Nielsen's heuristics), and compatibility issues (device-OS matrices). Validate output quality using a 5-dimension expert rubric with inter-annotator agreement.

Aim 2: Build an Issue-Spec-Aware Response Generation System

Implement a response generation module that uses RAG with 5 fixed input sources and is aware of the structured issue spec from Stage 3, producing user-facing replies that reference the specific issue and its status rather than generic templates. Compare against context-unaware baselines (RRGGen, CoRe).

Aim 3: Design and Evaluate a Dual-Objective RLHF Loop

Embed human oversight at three pipeline stages (classification verification, issue spec validation, final response review) and implement a progressive RLHF strategy (KTO to DPO to Constrained PPO) grounded in CMDP theory that jointly optimizes for response helpfulness and policy compliance, with feedback propagating backward to improve upstream stages.

6. Expected Outcomes

- 1. Structured Issue Database:** The pipeline produces a continuously growing database of standardized, schema-mapped issue specifications from raw reviews -- each with fields for issue type, reproduction steps, severity, affected component, and environment. This replaces ad-hoc manual triage with a structured, queryable knowledge base.
- 2. Faster Triage:** KG-based prioritization with graph centrality ranking significantly reduces developer triage time compared to manually reading raw review feeds. Measured via time from review intake to actionable response draft (in developer-hours) against the manual baseline.
- 3. Better Developer Insights:** The knowledge graph and hierarchical clustering provide developers with aggregated, deduplicated views of user complaints -- revealing temporal patterns (e.g., regressions after specific updates), cross-cutting issues, and device/OS-specific problem distributions that are invisible when reading reviews individually.
- 4. Improved User Response Quality:** Issue-spec-aware response generation produces replies that reference the specific problem and its status, replacing generic template responses. Dual-objective RLHF (CMDP-grounded) ensures responses are both helpful and policy-compliant, with measurable improvement over iterations.
- 5. Novel Benchmark Dataset:** The first dataset pairing app review clusters with expert-written structured issue specifications (200 to 300 clusters, 3+ annotators), enabling reproducible evaluation of future review-to-issue translation systems.
- 6. Theoretical Contribution:** Formal demonstration that IE cascade theory, human-AI complementarity, and CMDP provide a principled foundation for agentic review processing pipelines.
- 7. Multiple Publication Contributions:** The full system targets ICSE/FSE, the translation component and dataset targets ASE, the RLHF application targets EMNLP, and the KG-based triage targets the RE conference.

7. Proposed Methodological Steps and Architecture

7.1 System Architecture Overview (Scoped)

The ReviewAgent pipeline consists of five stages with human-in-the-loop (HITL) checkpoints integrated at three critical junctures. Based on advisor feedback, the scope focuses on Stages 1-3 + 4b + 5 as the core contribution. Stage 4a (agentic code resolution) is positioned as future work since it constitutes a separate research area with existing solutions (SWE-Agent, RepairAgent). Each stage is theoretically grounded:

- Stages 1-3: Justified by Information Extraction Cascade theory (progressive structuring)
- HITL Checkpoints: Justified by Human-AI Complementarity and Selective Prediction theory
- Stage 5 (RLHF): Justified by Constrained MDP theory (dual-objective optimization)

Stage 1: Intake with Human Verification

Raw app reviews undergo three parallel NLP tasks: (1) multi-label classification using a fine-tuned RoBERTa model trained on the MAALEJ and GUZMAN datasets, (2) aspect-based sentiment analysis to capture per-aspect polarity (e.g., 'Great UI but terrible battery drain' yields UI:positive, battery:negative), and (3) entity extraction to identify devices, OS versions, screens, and feature names. Ambiguous classifications (below a confidence threshold) are flagged for expert verification via selective prediction (Geifman & El-Yaniv, 2017), and corrections feed back into the training loop.

Reference papers: Kaur & Sahu (BERT-RCNN, 2023); Shah et al. (arXiv:2108.00663); Guzman & Maalej (2014); Geifman & El-Yaniv (2017); Bansal et al. (2019).

Stage 2: Unified Clustering-to-Issue Framework

This stage implements a three-layer process grounded in IE cascade theory:

1. Knowledge Graph Construction: Structured review objects become nodes (aspects, entities, reviews) and edges (sentiment, temporal) in a knowledge graph. [Ref: ReviewGraph (2023); KGCPN (2023)]
2. Hierarchical Clustering: Reviews are grouped first by aspect (coarse), then sub-clustered by specific complaint using semantic similarity and graph structure. [Ref: Villarroel et al., ICSE 2016 'CLAP'; Di Sorbo et al., FSE 2016 'SURF']
3. Standardized Issue Schema Mapping: Each cluster is mapped to a fixed schema containing fields for issue type, affected component, sentiment distribution, representative reviews, entities, temporal patterns, and a priority score computed via graph centrality (PageRank/betweenness). [Ref: Keertipati et al., EASE 2016]

Stage 3: Review-to-Issue Translation with Expert Validation

The core novel contribution. An LLM agent receives standardized cluster schemas and generates GitHub-issue-quality specifications grounded in established SE taxonomies:

1. Bug reports follow the Zimmermann template -- summary, steps to reproduce, expected vs. actual behavior, environment, severity. [Ref: Zimmermann et al., 2010; Chaparro et al., 2017]
2. Feature requests use user story format (As a [user], I want [feature], so that [benefit])
3. Performance complaints map to non-functional requirement categories per ISO/IEC 25010 (latency, memory, battery drain, network)
4. Usability issues align with Nielsen's heuristics (1994) -- visibility, feedback, error prevention, consistency

5. Compatibility issues generate device-OS-version matrices

Domain experts review each generated issue spec using a standardized 5-dimension rubric (completeness, accuracy, actionability, specificity, clarity) scored 1 to 5. Multiple experts rate independently; inter-annotator agreement is measured via Krippendorff's alpha. Low-scoring specs are returned to the LLM with dimension-level feedback for regeneration.

Stage 4b: Issue-Spec-Aware Response Generation

This module generates user-facing responses using RAG with 5 fixed input sources: (1) app's past review-response pairs from RRGGen 570K, (2) app changelog/release notes, (3) app FAQ/help documentation, (4) generated issue spec from Stage 3, and (5) similar past responses retrieved by semantic similarity. Responses are issue-spec-aware: they reference the specific issue and its status. A self-refinement loop (2 to 3 iterations) critiques and improves the draft for vagueness, unauthorized promises, and empathy. [Ref: Gao et al. (RRGen, 2019); CoRe (2020); Self-Improving Response Gen (ECNLP 2024)]

Stage 5: Dual-Objective Human Feedback (CMDP-Grounded)

Experts provide dimension-level feedback across two streams, formalized as a constrained MDP (Altman, 1999):

Stream 1 -- Quality (reward to maximize): Helpfulness, specificity, empathy, accuracy, and actionability.

Stream 2 -- Policy Compliance (constraint): Promise safety, information leakage, tone guidelines, and legal safety.

Progressive RLHF: KTO (Ethayarajh et al., 2024) for binary signals with small data, DPO (Rafailov et al., 2023) for paired preferences with growing data, Constrained PPO (Dai et al., 2023) for dual-objective optimization at scale.

Feedback propagates backward: corrections improve Stage 1 classification, rubric scores refine Stage 3 prompting/fine-tuning, and response quality scores drive the RLHF loop.

8. Experiment Design

Experiment 1: Review-to-Issue Translation Quality (RQ1)

Goal: Evaluate LLM-generated issue specs against human-written ones.

Setup: 100 review clusters from the KG. 3 experts write gold-standard issue specs. LLM generates specs for the same clusters.

Conditions: (a) LLM with taxonomy grounding (full system), (b) LLM without taxonomy grounding (free-form), (c) Raw review summary without structuring (lower bound), (d) Human-written issue specs (upper bound).

Evaluation: 3 independent raters score all specs on the 5-dimension rubric (1-5 each). Inter-annotator agreement via Krippendorff's alpha.

Statistical test: Paired Wilcoxon signed-rank test on rubric scores between conditions.

Experiment 2: Coupled vs. Uncoupled Response Generation (RQ2)

Goal: Does issue-spec-aware response generation outperform context-unaware baselines?

Setup: For the same 100 clusters, generate responses using: (a) RRGGen (no issue context), (b) CoRe (contextual but no structured issue), (c) ReviewAgent 4b WITHOUT issue spec (RAG only), (d) ReviewAgent 4b WITH issue spec (full system).

Evaluation: Automatic -- BLEU, ROUGE-L, BERTScore. Human -- 3 raters score helpfulness, specificity, empathy, accuracy (1-5 each).

Statistical test: Friedman test across 4 conditions, post-hoc Nemenyi pairwise comparisons.

Experiment 3: Dual-Objective vs. Single-Objective RLHF (RQ3)

Goal: Does decomposing feedback into quality + compliance outperform single-objective?

Setup: Train 3 variants -- (a) Single-objective KTO (binary good/bad), (b) Single-objective DPO (paired preferences on overall quality), (c) Dual-objective Constrained PPO (quality reward + compliance constraint).

Training data: Start with 500 rated responses, expand to 2000 over 3 iterations.

Evaluation: Human preference win rate (pairwise), safety violation rate, per-dimension rubric improvement over iterations.

Statistical test: Bradley-Terry model for preference data; McNemar's test for safety violation rates.

9. Ablation Studies

ID	What's Removed	What It Tests	Measured By
A1	No KG (skip Stage 2, feed raw classified reviews to Stage 3)	Value of knowledge graph clustering	Issue spec rubric scores
A2	No hierarchical clustering (flat clustering instead of aspect -> sub-cluster)	Value of two-level hierarchy	Cluster purity, NMI; downstream issue spec quality
A3	No taxonomy grounding (LLM generates free-form issues)	Value of literature-grounded templates	Issue spec rubric scores (completeness, actionability)
A4	No HITL at Stage 3 (skip expert validation)	Value of human checkpoint	End-to-end response quality
A5	No RAG (generate responses from issue spec + LLM only)	Value of retrieval augmentation	Response BLEU, human eval
A6	No issue spec in response gen (RAG but no structured issue)	Value of coupling Stages 3 and 4b	Response specificity, accuracy scores
A7	Single-stream feedback (merge quality and compliance)	Value of dual-objective decomposition	Preference win rate, safety violation rate

10. Experimental Variables

10.1 Independent Variables (Manipulated)

Variable	Levels	Experiment
Issue generation method	Human / LLM with taxonomy / LLM free-form / No structuring	Exp 1
Response generation context	No context / RAG only / Issue spec only / RAG + Issue spec	Exp 2
RLHF objective	Single (KTO) / Single (DPO) / Dual (Constrained PPO)	Exp 3
KG presence	With KG clustering / Without KG	Ablation A1
HITL checkpoint	With expert validation / Without	Ablation A4

Taxonomy grounding	Grounded in SE literature / Free-form	Ablation A3
--------------------	---------------------------------------	-------------

10.2 Dependent Variables (Measured)

Variable	Metric	Scale
Issue spec quality	5-dimension rubric scores	1-5 per dimension
Inter-annotator agreement	Krippendorff's alpha	0-1
Response quality (automatic)	BLEU, ROUGE-L, BERTScore	0-1
Response helpfulness	Human rating	1-5
Response specificity	Human rating	1-5
Response empathy	Human rating	1-5
Safety violation rate	% of responses with compliance issues	0-100%
Preference win rate	Pairwise human preference	%

10.3 Control Variables (Held Constant)

- Same LLM backbone across all conditions (e.g., GPT-4 or Llama 3)
- Same 100 review clusters for all experiments
- Same 3 expert raters across conditions
- Same rubric definitions and scoring guidelines
- Same RAG corpus configuration (5 fixed sources)

11. Methodological Steps

Step	Activity	Reference Papers	Output
Step 1	Collect and preprocess datasets (RRGen 570K, MAALEJ 4K, GUZMAN); identify open-source Android apps with both Play Store reviews and GitHub repos	Gao et al. (RRGen, 2019); Maalej et al. (2016)	Curated multi-source dataset
Step 2	Fine-tune RoBERTa for multi-label review classification; implement aspect-based sentiment analysis and entity extraction with HITL verification via selective prediction	Kaur & Sahu (2023); Guzman & Maalej (2014); Geifman & El-Yaniv (2017); Bansal et al. (2019)	Stage 1 classifier with active learning loop
Step 3	Build knowledge graph; implement hierarchical clustering and standardized schema mapping with priority ranking via graph centrality	ReviewGraph (2023); KGCPN (2023); Villarroel et al. (ICSE 2016); Di Sorbo et al. (FSE 2016); Keertipati et al. (2016)	Unified clustering-to-issue framework
Step 4	Develop LLM-based review-to-issue translation with taxonomy-grounded prompting; construct gold-standard dataset (200-300 clusters, 3+ expert annotators)	Zimmermann et al. (2010); Chaparro et al. (2017); Nielsen (1994); ISO/IEC 25010	Issue translation module + benchmark dataset
Step 5	Implement issue-spec-aware response generation with RAG (5 fixed sources) and self-refinement loop	Gao et al. (RRGen, 2019); CoRe (2020); Self-Improving (ECNLP 2024)	Stage 4b
Step 6	Build dual-objective RLHF pipeline (KTO, DPO, Constrained PPO) grounded in CMDP theory with backward feedback propagation	Ethayarajh et al. (2024); Rafailov et al. (2023); Dai et al. (2023); Altman (1999)	Stage 5 + iterative improvement loop
Step 7	End-to-end evaluation: 3 experiments + 7 ablation studies; expert rubric evaluation with inter-annotator agreement; statistical tests	Zimmermann et al. (2010); Chaparro et al. (2017)	Experimental results for RQ1-RQ3

12. RAG Input Materials (Fixed)

RAG Input Source	Purpose	Availability
App's past review-response pairs (RRGen 570K)	Learn tone, style, common acknowledgments	Available
App changelog / release notes	Reference specific versions and fixes	Scrapable from Play Store
App FAQ / help documentation	Provide accurate workarounds	App-specific, manual collection
Generated issue spec from Stage 3	Make response issue-aware and specific	Pipeline output
Similar past responses (semantic similarity)	Template-level guidance	From RRGen dataset

13. Dataset Strategy

Dataset	What It Provides	Pipeline Stage	Reference
RRGen (~570K pairs)	Google Play review-response pairs	Stages 1, 4b	Gao et al. (ASE 2019)
MAALEJ (~4K labeled)	Multi-label classified app reviews (bug, feature, rating, UX)	Stage 1	Maalej et al. (2016)
GUZMAN	Aspect-level annotations for app reviews	Stage 1	Guzman & Maalej (2014)
Open-source Android apps with GitHub repos	Ground-truth: reviews to issues to code fixes	Stage 3	Manual identification
Custom gold-standard (to be built)	Expert-written structured issue specs paired with review clusters	Stage 3 eval	Novel contribution
SWE-bench	Benchmark for agentic code resolution	Future work (4a)	Jimenez et al. (2024)

14. Evaluation Metrics (Finalized)

14.1 Gold Standard Evaluation (Stage 3 Output)

Metric	What It Measures	How
Rubric scores (5 dimensions)	Issue spec quality	3 experts, 1-5 scale per dimension
Krippendorff's alpha	Annotator agreement	Across 3+ raters
Completeness ratio	% of required fields filled	Automated check against schema
BERTScore vs. human specs	Semantic similarity to expert output	BERTScore (LLM vs. human)

14.2 Per-Stage Metrics

Stage	Primary Metric	Secondary Metric	Baseline
Stage 1: Classification	Macro F1, per-label F1	Confidence calibration (ECE)	BERT-RCNN
Stage 2: Clustering + Schema Mapping	Cluster purity, NMI, schema mapping correctness	Dedup precision/recall	K-means
Stage 3: Translation	Rubric scores + Krippendorff's alpha	BERTScore vs. human specs	Human-written issues
Stage 4b: Response	BLEU, ROUGE-L, BERTScore	Human eval (helpfulness, specificity, empathy)	CoRe, RRGGen
Stage 5: RLHF	Preference win rate	Safety violation rate, per-dim improvement	Single-objective tuning
End-to-end	% reviews reaching validated response	Time: intake to response draft vs. manual triage (developer-hours)	Manual triage (human-only)

15. Reference Papers per Methodology Step

Methodology Step	Reference Papers	What They Justify
Step 1: Dataset collection	Gao et al. (RRGen, 2019); Maalej et al. (2016)	RRGen 570K; MAALEJ classification
Step 2: RoBERTa classification	Kaur & Sahu (2023); Shah et al. (2021)	Transfer learning for reviews
Step 2: Aspect sentiment	Guzman & Maalej (2014); NLP survey (2024)	Aspect extraction methodology
Step 2: HITL classification	Geifman & El-Yaniv (2017); Bansal et al. (2019)	Selective prediction; human-AI teams
Step 3: KG construction	ReviewGraph (2023); KGCPN (2023)	KG for review representation
Step 3: Hierarchical clustering	Villarroel et al. (ICSE 2016); Di Sorbo et al. (FSE 2016)	Review clustering for prioritization
Step 3: Graph centrality	Keertipati et al. (EASE 2016)	PageRank for issue ranking
Step 4: Issue translation	Zimmermann et al. (2010); Chaparro et al. (2017)	Bug report structure and quality
Step 4: Issue taxonomies	Nielsen (1994); ISO/IEC 25010:2011	Usability heuristics; NFR model
Step 5: RAG response gen	RRGen (2019); CoRe (2020); Self-Improving (2024)	RAG and self-refinement
Step 6: KTO	Ethayarajh et al. (2024)	Binary feedback RLHF
Step 6: DPO	Rafailov et al. (2023)	Paired preference RLHF
Step 6: Constrained PPO	Dai et al. (2023); Altman (1999)	Dual-objective optimization
Theory: IE cascade	Hearst (1999); Sarawagi (2008)	Progressive structuring
Theory: Human-AI	Kamar (2016); Bansal et al. (2019)	HITL checkpoint placement
Theory: CMDP	Altman (1999); Dai et al. (2023)	Dual-objective RLHF